



National and Kapodistrian University of Athens (NKUA)
MSc in Data Science and Information Technologies (DSIT)
Course: Algorithms in Structural Bioinformatics (2025–2026)

Student: *Konstantinos Kritsotakis*

Registration Number: *7115152500007*

Background: *Chemical Engineer, National Technical University of Athens (NTUA)*

Graduate Student: *MSc DSIT – Specialization: Big Data & Artificial Intelligence*

1. Sequence comparison algorithms

(a)

The toy dataset was generated exactly according to the Python script provided in **Annex A**. First, a **reference protein sequence** BASE of length **200 amino acids** was created by random sampling from the standard 20-amino-acid alphabet:

$$\text{ALPHABET}=\{\text{A,C,D,E,F,G,H,I,K,L,M,N,P,Q,R,S,T,V,W,Y}\}$$

with `random.seed(0)` to ensure reproducibility. From this reference sequence, a **motif** of length **20 aa** was defined as:

$$\text{MOTIF} = \text{BASE}[80:100]$$

and a **fragment** of length **40 aa** was defined as:

$$\text{fragment} = \text{BASE}[80:120]$$

The **database** was then constructed as a FASTA file containing **100 protein sequences**, all of length **200 aa**, divided into four classes:

1. **25 strong homologs** of the reference sequence, generated as `mutate(BASE, 0.15)`, i.e. by mutating each position independently with probability **0.15**.
2. **25 weak homologs**, generated as `mutate(BASE, 0.50)`, i.e. with mutation probability **0.50** per position.
3. **25 random sequences**, generated independently by `random_seq(200)`.
4. **25 motif-containing sequences**, generated by first creating a random sequence of length 200 and then replacing a segment of length 20 at a random position `randint(40,160)` with MOTIF.

Thus, the database contains:

- 25 strong homologs,
- 25 weak homologs,
- 25 random proteins,
- 25 proteins containing the reference motif,

for a total of **100 sequences** written to db.fasta.

The **query set** was also generated by the same script and contains **15 queries** divided into five classes:

1. **3 strong queries**, generated as mutate(BASE, 0.05).
2. **3 weak queries**, generated as mutate(BASE, 0.50).
3. **3 random queries**, generated as random_seq(200).
4. **3 fragment queries**, generated as:

random_seq(80) + fragment + random_seq(80)

so that they contain a conserved **40-aa region** embedded in random sequence.

5. **3 motif queries**, generated as:

random_seq(80) + MOTIF + random_seq(80)

so that they contain the conserved **20-aa motif** embedded in random sequence.

The resulting queries were written to queries.fasta.

Therefore, the generated dataset provides a controlled benchmark including:

- highly similar full-length homologs,
- weak homologs,
- completely unrelated random sequences,
- sequences with only a conserved **local fragment**,
- and sequences containing only a short conserved **motif**.

This design is appropriate for comparing the behavior of exact and heuristic sequence comparison methods, since it includes both **global similarity cases** and **local similarity cases**.

Note

Following the Annex A script exactly, the database sequences are all **200 aa** long, and the strong, weak, random, and fragment queries are also **200 aa** long. The motif queries, however, are generated as random_seq(80) + MOTIF + random_seq(80), which yields sequences of length **180 aa**. This is the direct output of the provided script and was preserved unchanged.

(b)

1. Exact methods: Needleman–Wunsch and Smith–Waterman

In this part, the four programs were executed for all queries against the toy protein database generated in part (a): **ggsearch36** (Needleman–Wunsch, global alignment), **ssearch36** (Smith–Waterman, local alignment), **fasta36** (FASTA heuristic), and **blastp** (BLAST heuristic). Their runtime behavior can be understood by separating the **exact dynamic-programming methods** from the **heuristic database-search methods**.

1. Exact methods: Needleman–Wunsch and Smith–Waterman

The exact methods are based on **dynamic programming (DP)**.

- **Needleman–Wunsch** computes an optimal **global alignment** between two sequences.

- **Smith–Waterman** computes an optimal **local alignment** between two sequences.

For two sequences of lengths m and n , both methods fill a DP table of size $m \times n$. Each cell is computed from a constant number of neighboring cells, so the time required for one pairwise comparison is:

$$O(mn)$$

and the memory requirement of the standard implementation is also:

$$O(mn)$$

Thus:

- **ggsearch36** has runtime $O(mn)$ per query–database sequence pair.
- **ssearch36** has runtime $O(mn)$ per query–database sequence pair.

Since in this assignment all queries are searched against all database sequences, the total runtime for the exact methods is proportional to:

$$O\left(\sum_{q=1}^Q \sum_{d=1}^D m_q n_d\right)$$

where:

- $Q=15$ queries,
- $D=100$ database sequences,
- and sequence lengths are approximately 200 amino acids.

Therefore, in this toy dataset, the total work is on the order of:

$$15 \cdot 100 \cdot 200 \cdot 200 = 60,000,000$$

DP cell computations, up to constant factors.

This explains why **ssearch36** and **ggsearch36** are computationally the most expensive methods: they perform an exact optimization for every query–target pair.

2. Heuristic methods: FASTA and BLAST

The other two methods are **heuristic database-search algorithms**, designed to avoid full DP over the entire database whenever possible.

FASTA

FASTA begins from **short exact matches (k-tuples / seeds)** and uses them to identify promising diagonals in an implicit dot matrix. It then keeps only the best candidate regions and performs more expensive alignment refinement only there.

Its runtime is therefore much smaller than exhaustive DP in practice. The theory emphasizes that:

- the first seeding stage is essentially linear for fixed k ,
- only a small number of good diagonals are retained,
- the dominant later phases are **inversely proportional to k tup**.

Hence, FASTA does **not** perform a full $O(mn)$ dynamic-programming computation on every database sequence. Instead, it reduces the search space aggressively before refinement.

BLAST

BLAST uses a **seed-and-extend** strategy:

1. It identifies short fixed-length words (**k-tuples**) from the query.
2. It finds database hits above a threshold.
3. It extends them left and right while the score keeps improving.
4. It retains only **HSPs** (High-scoring Segment Pairs) above acceptance score.

According to the theory:

- **sensitivity is inversely proportional to k ,**
- **runtime is inversely proportional to k ,**
- and the bottleneck time is approximately proportional to:

$$\text{seed size} \times \text{database size}$$

Thus, BLAST is also far more efficient than exact DP for large-scale database searching, because it evaluates only regions supported by seed hits rather than all possible alignments.

3. Relative runtime behavior of the four programs

The four methods therefore fall into two categories:

Exact methods

- **ggsearch36 (Needleman–Wunsch):** $O(mn)$ per pair
- **ssearch36 (Smith–Waterman):** $O(mn)$ per pair

These are exact, but computationally expensive.

Heuristic methods

- **fasta36**
- **blastp**

These are much faster in practice because they avoid full dynamic programming on all query–database pairs.

Therefore, for large databases, the expected runtime ranking is:

$$\text{BLAST} \approx \text{FASTA} \ll \text{Smith–Waterman, Needleman–Wunsch}$$

up to implementation details and parameter settings.

4. Difference between Smith–Waterman and Needleman–Wunsch in runtime

Although **Smith–Waterman** and **Needleman–Wunsch** solve different problems (local vs global alignment), their standard DP runtime is of the same asymptotic order:

$$O(mn)$$

The difference is not in asymptotic complexity, but in the recurrence relation and in the biological interpretation:

- Needleman–Wunsch aligns the **entire lengths** of the two sequences.
- Smith–Waterman allows **0-restart**, so it finds the best **local** region.

Nevertheless, from a computational viewpoint, both still require filling an $m \times n$ table.

5. Difference between FASTA and BLAST in runtime

Both are heuristic local-search methods, but their strategies differ.

- **FASTA** starts from exact short matches, scores diagonals, keeps only a small number of candidates, and then refines them.
- **BLAST** also starts from short words, but then extends only seed-supported regions and retains only HSPs above threshold.

In both cases, the runtime is dominated not by full DP over all cells, but by:

- the number of seed hits,
- the seed size k ,
- the threshold / drop-off,
- and the database size.

Hence both are substantially faster than exact methods, especially on large databases.

6. Interpretation for the present assignment

In addition to the theoretical complexity analysis, the programs were timed on the toy dataset. The observed runtimes were:

- **blastp = 0.070 s**
- **ggsearch36 = 0.195 s**
- **ssearch36 = 0.280 s**
- **fasta36 = 10.554 s**

The local BLAST database construction step, **makeblastdb**, required **0.442 s** and was treated as a preprocessing step rather than as one of the four search methods.

Empirically, the runtime ranking on this dataset was:

$$\text{BLASTP} < \text{GGSEARCH36} < \text{SSEARCH36} \ll \text{FASTA36}$$

7. Interpretation

These empirical timings should be interpreted with caution, because this is a very small benchmark. In such a setting, practical runtime is influenced not only by asymptotic complexity, but also by:

- constant factors,
- implementation details,
- startup overhead,
- input/output overhead,
- and program-specific heuristics and filtering strategies.

For this reason, the observed runtime order on a toy dataset does not necessarily mirror the asymptotic expectations perfectly. In particular, although **FASTA** is a heuristic method, it was slower than the exact DP methods in this specific experiment. This does not contradict the theoretical analysis; it simply reflects the fact that runtime on small datasets is often dominated by implementation-specific overhead rather than by asymptotic growth.

8. Final conclusion

To summarize:

- **Needleman–Wunsch (ggsearch36)**: exact global alignment, runtime $O(mn)$ per sequence pair.
- **Smith–Waterman (ssearch36)**: exact local alignment, runtime $O(mn)$ per sequence pair.
- **FASTA (fasta36)**: heuristic local database search based on k-tuples, diagonals, and restricted refinement; designed to reduce the search space compared with exact DP.
- **BLAST (blastp)**: heuristic seed-and-extend local database search; runtime depends strongly on word size k , thresholding, and database size, and is designed for efficient large-scale database search.

Therefore, the exact DP methods provide optimal alignments but at higher theoretical computational cost, whereas FASTA and BLAST trade a small amount of sensitivity for improved scalability. This is why heuristic methods are preferred for large biological databases, even though, on this small toy dataset, the measured runtimes are strongly affected by implementation and overhead effects.

(c)

To answer this question, the exact dynamic-programming methods were used as the reference standard. More specifically, for each query, the set of sequences identified by the DP methods was defined as the union of the hits returned by ssearch36 (Smith–Waterman) and ggsearch36 (Needleman–Wunsch). FASTA and BLAST were then compared against this DP reference set.

For each query q , let

$$DP_gold(q) = hits_ssearch36(q) \cup hits_ggsearch36(q).$$

Then the sequences missed by a heuristic method $H \in \{\text{FASTA}, \text{BLAST}\}$ were defined as

$$missed_H(q) = DP_gold(q) \setminus hits_H(q).$$

The corresponding percentage of missed sequences was computed as

$$\text{missed percentage} = |missed_H(q)| / |DP_gold(q)| \times 100.$$

Two complementary analyses were performed.

First, an overall analysis was carried out using all queries and all database sequence categories. In this setting, FASTA failed to identify 8.82% of the sequences detected by the DP methods, whereas BLAST failed to identify 10.17%.

Second, a clean weak-homolog analysis was performed in order to focus specifically on the most biologically relevant difficult case. In this analysis, only the weak queries (query3–query5) and only the weak homolog database sequences (prot25–prot49) were considered. Under this stricter criterion, FASTA failed to identify 0% of the weak homologs detected by the DP methods, whereas BLAST failed to identify 4%.

These results are consistent with the theoretical behavior of the methods. The DP methods are exact and therefore provide the most complete recovery of homologous sequences. By contrast, FASTA and BLAST are heuristic methods that accelerate database search by reducing the search space through seeds, diagonals, and seed-and-extend strategies. As a consequence, they may miss some valid matches, especially weak homologs, whose similarity signal is less pronounced than in the strong-homolog case.

In conclusion, FASTA and BLAST achieve faster searches than the exact DP methods, but this speedup comes at the cost of reduced sensitivity. In the present dataset, this loss of sensitivity is small overall, but it remains measurable. Under the stricter clean weak-homolog analysis, FASTA recovered all weak homologs found by the DP methods, while BLAST missed a small fraction (4%). This confirms that heuristic methods are generally effective, but may still fail to recover some weaker homologous sequences compared with exact dynamic-programming approaches.

(d)

To quantify the accuracy of the four methods, two complementary analyses were performed.

First, the top-ranked sequence returned for each query was compared across the four programs: **BLAST**, **FASTA**, **ssearch36** (Smith–Waterman), and **ggsearch36** (Needleman–Wunsch). Second, for each pair of methods and for each query, the two top-5 result sets were compared using **Spearman’s rank correlation**, after assigning rank 6 to any item absent from a method’s top-5 list, exactly as required in the statement. The correlation values were then averaged over all queries and reported in a 4 x 4 table.

1. Comparison of the top sequence returned for each query

The top-1 comparison shows that the methods agree strongly on the easier cases, but diverge more on the more ambiguous ones.

For the **strong homolog queries**, agreement was high. For example, for query1 all four methods returned prot21 as the top hit, while for query0 and query2 FASTA, ssearch36, and ggsearch36 agreed with each other, whereas BLAST returned a different top sequence. This indicates that for clear homologous cases the methods behave similarly, although BLAST may alter the exact top ranking because of its heuristic seed-and-extend strategy.

For the **weak homolog queries**, FASTA and ssearch36 were again highly consistent, and ggsearch36 often agreed or differed only slightly, while BLAST was more variable. For instance, for query3 BLAST returned prot7, whereas FASTA, ssearch36, and ggsearch36 all returned prot20. For query5, all four methods differed to some extent (prot4, prot14, prot14, prot22), showing that weak cases are much less stable and therefore more informative about differences in search strategy.

For the **random queries**, disagreement was strongest. This is expected, because unrelated sequences do not provide a strong biological signal, so small differences in scoring and search strategy can change the ranking substantially. For example, for query7, all four methods returned different top hits.

For the **fragment queries**, agreement improved again. In particular, for query9 and query11, all four methods returned the same top sequence (prot10), while for query10 BLAST returned prot10 and the other three methods returned prot13. This is consistent with the fact that fragment queries contain a strong local conserved region, which is especially favorable for local-search methods.

For the **motif queries**, disagreement increased again. For example, for query12, BLAST, FASTA, and ssearch36 agreed on prot95, whereas ggsearch36 returned prot76. For query14, BLAST returned prot18, FASTA and ssearch36 returned prot75, and ggsearch36 returned prot98. This is biologically reasonable, because motif queries contain only a short-conserved region embedded in otherwise random sequence, so global ranking is more sensitive to method design.

Overall, the top-1 comparison leads to the following conclusions:

- **FASTA and ssearch36 are the most similar in top-1 behavior**, agreeing on almost all queries.
- **ggsearch36** is also close to them, but differs more often on motif and random cases, which is expected because it is a **global** method.
- **BLAST** is the most variable with respect to the exact methods, especially on weak, random, and motif queries, which is consistent with its heuristic nature.
- Agreement is highest for **strong homolog** and some **fragment** queries, and lowest for **random** and **motif** queries.

2. Pairwise top-5 comparison using Spearman's rank correlation

For each query and each pair of methods, the union of the two top-5 result sets was formed. Each item was assigned its rank in each method, and any item missing from a method's top-5 list was assigned rank 6. Spearman's rank correlation was then computed for the two induced rankings. The final reported value for each pair of methods is the average correlation over all queries.

The resulting average 4 x 4 Spearman correlation matrix is:

Method	BLAST	FASTA	SSEARCH	GGSEARCH
BLAST	1.000000	0.382937	0.354762	0.371190
FASTA	0.382937	1.000000	0.849048	0.569365
SSEARCH	0.354762	0.849048	1.000000	0.544444
GGSEARCH	0.371190	0.569365	0.544444	1.000000

3. Interpretation of the Spearman matrix

The highest off-diagonal correlation is between **FASTA** and **ssearch36**:

$$\rho=0.849048$$

This shows that these two methods produce very similar top-5 rankings on average. This is a strong result and indicates that FASTA, although heuristic, approximates the ranking behavior of exact local alignment very well on this dataset.

The next strongest correlations involve **ggsearch36**:

- FASTA vs GGSEARCH: **0.569365**
- SSEARCH vs GGSEARCH: **0.544444**

These values are moderate, which is expected. ggsearch36 performs **global alignment**, whereas FASTA and ssearch36 are fundamentally more favorable to **local similarity**, so their rankings need not coincide, especially for fragment and motif queries.

The correlations involving **BLAST** are the lowest:

- BLAST vs FASTA: **0.382937**
- BLAST vs SSEARCH: **0.354762**
- BLAST vs GGSEARCH: **0.371190**

Thus, BLAST shows the greatest deviation from the other three methods in ranking behavior. This does not necessarily mean that BLAST is inaccurate in an absolute sense; rather, it indicates that its heuristic search and ranking strategy lead to a substantially different ordering of the top candidates, particularly in the harder cases.

4. Final conclusion

The accuracy analysis shows that the four methods are not equally similar in behavior.

- At the **top-1 level**, the methods agree strongly on clear homologous cases, especially strong homolog and some fragment queries.
- The agreement decreases on more difficult cases, especially weak, random, and motif queries.
- **FASTA and ssearch36** are the most consistent pair both in top-1 behavior and in top-5 ranking, indicating that FASTA is the heuristic method that most closely reproduces the behavior of exact local alignment on this dataset.
- **ggsearch36** is moderately correlated with FASTA and ssearch36, which is consistent with the difference between **global** and **local** alignment objectives.
- **BLAST** is the least correlated with the other methods and shows the greatest variability in both top-1 and top-5 ranking, reflecting its distinct heuristic design.

In summary, the results indicate that **FASTA is the heuristic method most similar to exact local alignment**, **ssearch36 provides the most stable exact local reference**, **ggsearch36 behaves differently when global similarity is emphasized**, and **BLAST is the most divergent in ranking behavior**, especially in biologically ambiguous cases.

(e)

To evaluate BLAST sensitivity, the search was repeated with two different seed lengths, using `-word_size 2` and `-word_size 7`. The purpose was to assess how seed length affects both runtime and the detection of strong and weak similarities.

In BLAST, the word size determines the length of the exact seed required to initiate extension. A smaller word size makes BLAST more sensitive, because shorter exact matches are easier to find, even between weakly related sequences. A larger word size is more restrictive, because it requires a longer exact seed before extension begins. Therefore, decreasing the word size is expected to increase sensitivity but also increase runtime.

This expectation was confirmed by the experiments. The measured runtimes were:

- `blastp -word_size 2`: **0.733726 s**
- `blastp -word_size 7`: **1.197190 s**

Thus, in this particular toy dataset, the run with `-word_size 7` was slower than the run with `-word_size 2`. Although the general theoretical expectation is that larger seeds reduce the search space and tend to accelerate the search, empirical runtime on a small benchmark can be strongly influenced by implementation details, hit distribution, overhead, and the internal extension/filtering behavior of BLAST. Therefore, the sensitivity conclusions are more important than the raw runtime difference in this small-scale setting.

Detection of strong similarities

For **strong homolog queries**, both seed lengths performed well. With `-word_size 2`, the average number of strong-database hits per strong query was **25.0**, and with `-word_size 7` it was also **25.0**. Thus, all strong homologs were recovered in both settings. This is expected, because strong similarities contain many exact short words and are easy for BLAST to detect regardless of seed size.

Detection of weak similarities

The effect of seed length is much clearer for **weak homolog queries**.

For weak queries:

- with `-word_size 2`, the average number of **weak_db** hits per query was **20.0**
- with `-word_size 7`, the average number of **weak_db** hits per query dropped to **7.0**

This is a substantial reduction in sensitivity. It shows that increasing the seed length makes BLAST much less effective at recovering weak homologs. This is biologically reasonable, because weakly related sequences often do not share many long exact words, even if they are still meaningfully related overall.

Detection of local similarities

The same trend appears for queries dominated by **local conserved regions**.

For **fragment queries**:

- with `-word_size 2`, the average number of **weak_db** hits was **20.0**
- with `-word_size 7`, the average number of **weak_db** hits fell to **0.0**

Thus, the larger seed length essentially eliminated weak-homolog recovery for fragment queries.

For **motif queries**:

- with `-word_size 2`, the average number of **weak_db** hits was **20.0**
- with `-word_size 7`, the average number of **weak_db** hits decreased to **13.0**

Again, the shorter seed was more sensitive

Overall interpretation

The results show the expected sensitivity trade-off very clearly:

- **shorter seeds** improve the detection of weaker and more local similarities;
- **longer seeds** preserve detection of strong homologs, but lose many weak matches.

In particular:

- strong homologs remained fully detectable with both seed lengths;
- weak homolog detection deteriorated strongly when the seed length increased from 2 to 7;
- local similarity cases (fragment and motif queries) were also more sensitive to seed length, especially fragment queries.

Therefore, the main biological conclusion is that **-word_size 2 is substantially more sensitive than -word_size 7 for weak and local similarities**, whereas both settings are adequate for strong similarities.

Final conclusion

BLAST shows the expected sensitivity–specificity trade-off with respect to seed length. A smaller seed length (`-word_size 2`) allows the detection of more weak homologs and local similarities, because shorter exact words are sufficient to trigger extension. A larger seed length (`-word_size 7`) is more restrictive and therefore misses many weaker relationships, even though strong homologs are still detected reliably. In this dataset, the shorter seed length was clearly preferable when sensitivity was the main objective, especially for weak homologs and partial local matches.

2. *Structural comparison of protein structures*

(a)

I used the segment consisting of **residues 50 to 99** from both protein structures, **6LU7** and **2AMD**, and kept only the **C α atom** of each residue. Thus, I obtained **50 corresponding 3D points** from each structure. The selected chain in both structures was **chain A**.

Let the C α coordinates from 6LU7 be $x_1, \dots, x_{50} \in \mathbb{R}^3$, and the corresponding C α coordinates from 2AMD be $y_1, \dots, y_{50} \in \mathbb{R}^3$.

The optimal rigid alignment was computed as follows.

First, I computed the centroids:

$$x_c = \frac{1}{50} \sum_{i=1}^{50} x_i, \quad y_c = \frac{1}{50} \sum_{i=1}^{50} y_i$$

The numerical values are:

$$x_c = (-16.737160, 31.663920, 70.607000)$$

$$y_c = (100.637900, 25.127940, 18.419440)$$

Then I centered the coordinates:

$$\tilde{x}_i = x_i - x_c, \quad \tilde{y}_i = y_i - y_c$$

If X and Y are the corresponding 50×3 matrices of centered coordinates, then the covariance matrix is:

$$H = X^T Y$$

Numerically,

$$H = \begin{bmatrix} -625.1980598 & 787.36735852 & -1120.43075948 \\ -1506.1045034 & 925.81800176 & 2746.95443376 \\ 2549.0638050 & 745.71644300 & -3738.71657700 \end{bmatrix}$$

Next, I computed the singular value decomposition:

$$H = U \Sigma V^T$$

with singular values:

$$\Sigma = \text{diag}(5536.0624814052, 1524.6054007325, 998.4212433730)$$

The optimal rotation is:

$$Q = UV^T$$

and since $\det(Q) = 1$, no reflection correction was needed. The resulting rotation matrix is:

$$Q = \begin{bmatrix} -0.7228152395 & 0.4049491249 & -0.5599592269 \\ -0.0696049338 & 0.7635274050 & 0.6420132826 \\ 0.6875269324 & 0.5030329095 & -0.5237027871 \end{bmatrix}$$

with

$$\det(Q) = 1.0000000000$$

The optimal translation vector is:

$$t = y_c - Qx_c$$

which gives:

$$t = (115.2547901268, -45.5439514256, 50.9757771554)$$

Therefore, the optimal rigid transformation that maps the segment of 6LU7 onto the segment of 2AMD is:

$$x_{\text{aligned}} = Qx + t$$

Finally, the coordinate RMSD of the aligned segments is:

$$\text{c-RMSD} = \sqrt{\frac{1}{50} \sum_{i=1}^{50} \|Qx_i + t - y_i\|^2}$$

and the computed value is:

$$\text{c-RMSD} = 15.4246077130 \text{ \AA}$$

(b)

For the same segments used in part (a), I considered **residues 50–99** from **chain A** of **6LU7** and **chain A** of **2AMD**, and used the corresponding **50 C α atoms** in each structure.

To compute the **distance RMSD (d-RMSD)**, I used **all pairwise C α distances** within each segment. For a set of $n = 50$ atoms, the number of all unordered pairs is

$$k = \binom{50}{2} = \frac{50 \cdot 49}{2} = 1225$$

So, the total number of pairwise C α distances used is

$$1225$$

Let d_{ij} be the Euclidean distance between the C α atoms of residues i and j in the segment of 6LU7, and let d'_{ij} be the corresponding distance in the segment of 2AMD, for all pairs with $50 \leq i < j \leq 99$. The d-RMSD is then defined as

$$\text{d-RMSD} = \sqrt{\frac{1}{k} \sum_{i < j} (d_{ij} - d'_{ij})^2}$$

with $k = 1225$.

Using all 1225 pairwise C α distances, I obtained:

$$\text{d-RMSD} = 0.2638633605 \text{ \AA}$$

Therefore, the d-RMSD of the segments consisting of residues 50–99 is **0.2639 \AA**, computed from **1225 pairwise C α distances**.

(c)

Using the same segments as in parts (a) and (b), namely **residues 50–99** from **chain A** of **6LU7** and **chain A** of **2AMD**, I considered the **50 corresponding C α atoms** in each structure. The exact d-RMSD from part (b), computed from all pairwise C α distances, is

$$\text{d-RMSD}_{\text{exact}} = 0.2638633605 \text{ \AA}.$$

Since there are 50 atoms in each segment, the total number of pairwise distances is

$$\binom{50}{2} = 1225.$$

I then approximated the d-RMSD by using only subsets of these 1225 distances, in two different ways:

1. by selecting the **shortest distances**, and
2. by selecting **randomly chosen distances**.

In both cases, I used subsets of size **500**, **200**, and **100**, and compared the approximate values with the exact d-RMSD.

The approximation formula is

$$\text{d-RMSD}_{\text{approx}} = \sqrt{\frac{1}{m} \sum (d_{ij} - d'_{ij})^2},$$

where m is the number of selected distances, and d_{ij} , d'_{ij} are the corresponding C α –C α distances in the two structures.

1. Approximation using the shortest distances

For this strategy, the pairwise distances were sorted from smallest to largest, and the first m distances were used.

Subset size	Approximate d-RMSD (\AA)	Absolute error (\AA)	Relative error
500	0.2504132227	0.0134501378	5.0974%
200	0.1862655110	0.0775978494	29.4083%
100	0.1497983524	0.1140650080	43.2288%

These results show that the approximation based on the shortest distances is only reasonably accurate when a relatively large subset is used. With 500 distances, the result is still close to the exact d-RMSD, but for 200 and especially 100 distances the approximation deteriorates substantially. This indicates that the shortest distances capture mainly **local geometric information** and do not represent the **overall internal geometry** of the segment sufficiently well.

2. Approximation using randomly selected distances

For the second strategy, I selected m distances uniformly at random from the 1225 pairwise distances.

Subset size	Approximate d-RMSD (Å)	Absolute error (Å)	Relative error
500	0.2627923228	0.0010710377	0.4059%
200	0.2591278353	0.0047355252	1.7947%
100	0.2714475219	0.0075841615	2.8743%

In this case, the approximation is excellent for all three subset sizes. Even with only 100 randomly selected distances, the approximate d-RMSD remains very close to the exact value. This suggests that random subsets provide a more balanced sample of the entire distance distribution and therefore preserve the global structural information much better than using only the shortest distances.

3. Stability of the random approximation over 200 trials

To evaluate how stable the random approximation is, I repeated the random selection **200 times** for each subset size. The results are summarized below.

Subset size	Mean d-RMSD (Å)	Standard deviation	Minimum (Å)	Maximum (Å)	Mean relative error
500	0.2637659995	0.0085298679	0.2404832395	0.2843981019	0.0369%
200	0.2633135122	0.0161112448	0.2184638347	0.3071320427	0.2084%
100	0.2619495828	0.0219224201	0.2036742116	0.3233855482	0.7253%

The mean approximation is extremely close to the exact d-RMSD for all subset sizes, which confirms that random selection is, on average, a very accurate approximation strategy. At the same time, the standard deviation increases as the subset size decreases, meaning that smaller random subsets are less stable and more sensitive to the particular sample drawn. Thus, while 100 random distances can still provide a very good approximation, the variability across trials is higher than for 500 distances.

4. Comparison and conclusion

The results clearly show that the two approximation strategies behave very differently.

- Using the **shortest distances** gives a biased approximation, because these distances emphasize only very local structure. As a result, accuracy drops rapidly when the subset size becomes smaller.
- Using **randomly selected distances** gives a much better approximation to the exact d-RMSD, even for small subsets, because it samples distances across the entire segment and therefore reflects the global geometry more faithfully.

- Larger subsets are generally more reliable than smaller ones, but in this case random subsets remain highly accurate even with only 100 distances.

Therefore, for this particular pair of segments, **randomly selected pairwise C α distances provide a much more accurate approximation of the exact d-RMSD than the shortest distances**, especially when the number of selected distances is small.